Stream-Aware, Leakage-Safe Build Failure Prediction for Continuous Integration: Pre-Registered Evidence from Travis CI and GitHub Actions

Author Name
Department, University
City, Country
email@example.com

***Abstract*—Predicting which continuous integration (CI) builds will fail lets teams spend a fixed review budget where it matters, but published results are easy to inflate: post-build fields leak the outcome, random train/test splits let the future predict the past, and pooled metrics reward telling repositories apart rather than ranking a repository's own builds. We study build failure prediction under a strict protocol: only information available when a build is triggered, history that counts a prior build only once it has *finished*, chronological walk-forward evaluation, and macro *within-repository* AUC with paired repository-bootstrap confidence intervals. On 858,368 Travis CI builds from 947 projects, a leakage-safe Random Forest with repository history (Tier C) reaches 0.628 within-repository AUC, far below the 0.91 a random split suggests. Adding history for the build's own *stream* (its pull request or branch), in-flight context, and a learning-to-rank objective (Tier E) raises this to 0.698 (+0.070, every fold's 95% CI above zero), including +0.066 on *new breakages*. We then tested the frozen recipe on GitHub Actions under protocols hashed before data collection: on 30 repositories the gain replicated in direction (+0.055) but was inconclusive under the pre-registered rule; on 30 *new* repositories it was confirmed (0.911 vs. 0.763, +0.149, 5 of 5 folds). We deploy the model behind a GitHub webhook that scores each workflow run when it is requested, with a per-repository 15% review budget, and release a read-only public demonstration. A forward-looking test of the frozen deployed system is pre-registered.**

***Index Terms*—continuous integration, build failure prediction, data leakage, learning to rank, pre-registration, GitHub Actions, software analytics**

I. Introduction

Continuous integration (CI) runs a project's build and tests on every push and pull request [1]. Failed builds cost developer attention and compute, and a large literature asks whether failures can be predicted before a build runs [2]–[6]. A useful predictor would let a team review, prioritise, or defer the riskiest builds within a fixed budget.

Reported accuracy in this area is often optimistic for three avoidable reasons. First, *leakage* [7]: CI datasets such as TravisTorrent [8] mix pre-build fields with fields computed from the build log, such as test counts and durations, that encode the outcome. Second, *random splits*: when builds are shuffled, models learn a repository's future failure rate and use it to predict its past [9]. Third, *pooled metrics*: repositories differ widely in failure rate, so a model can earn a high pooled AUC simply by recognising risky repositories, which says little about ranking a single team's builds, the decision a team actually faces.

This paper makes the following contributions:

- A **leakage-safe protocol** for build failure prediction: an availability registry enforced at training and inference time, finish-time-safe history, chronological walk-forward folds, and macro within-repository AUC with paired repository-bootstrap CIs. We quantify how much the shortcuts inflate results on TravisTorrent (Section IV).
- **Tier E**, a model that adds history for the build's own *stream* (pull request or branch), in-flight context, and a learning-to-rank objective. It improves within-repository AUC from 0.628 to 0.698 on Travis CI, and the gain holds on new breakages (Section VII).
- Two **pre-registered external tests** on GitHub Actions. The first replicated in direction but was inconclusive under its rule; a confirmation on 30 new repositories met it decisively (Section VII-C).
- A **deployed system**, with live history verified identical to the training pass, alerts issued under a per-repository review budget that matches the evaluation metric, and a read-only public demonstration (Section VIII).
- A **pre-registered forward-looking test** of the frozen deployed system on data that did not exist when it was frozen (Section X).

II. Related Work

CI practice and failures. Hilton et al. document CI's costs and benefits in open source [1]; Rausch et al. characterise build failure causes [10]. TravisTorrent [8] made Travis CI build histories widely available, and later studies flag noise and heterogeneity in such data [11], [12].

Build outcome prediction. Hassan and Wang predict build outcomes from change features [2]; Ni and Li use cascaded classifiers to cut cost [3]; Xia and Li compare learners [4]. BuildFast [5] adds history-aware features, and Jin and Servant

TABLE I
DATASETS. "FAILURE RATE" IS PER BUILD OR RUN.

Dataset	Units	Repos	Fail. rate	Period
Travis CI (TravisTorrent)	858,368	947	19.5%	2011–2016
GHA test 1	67,980	30	8.9%	2025–2026
GHA confirmation	78,576	30	28.2%	Aug–Sep 2026
GHA combined	146,556	60	19.2%	2025–2026

study cost-efficient build selection [6]. Common to most of this work are pooled metrics and splits or history features that ignore build finish times; history that ignores finish times can include builds that had not finished when the current build was triggered.

Methodology. We follow the leakage taxonomy of Kaufman et al. [7] and time-respecting evaluation [9], use bootstrap CIs [13], and adopt pre-registration [14] to keep confirmatory tests honest. The models use Random Forests [15], LightGBM [16] with the XE-NDCG ranking loss [17], Platt scaling [18], and SHAP explanations [19].

III. DATA

Table I summarises the three datasets.

Travis CI. The TravisTorrent export [8] has 3.56M job rows. We collapse them to **858,368 builds** from **947 projects** (April 2011 to August 2016), asserting that every pre-build field is constant across a build's jobs. A build fails if any of its jobs failed (19.5% of builds). A build's finish time is its start time plus its longest job's duration, with a per-repository 95th-percentile fallback for missing durations.

GitHub Actions. The unit is a workflow run. We keep first attempts of `push` and `pull_request` runs; `success` is a pass, and `failure` and `startup_failure` are failures. A run's finish time is its `updated_at`, and runs whose completion time is unknown (over 35 days) are dropped. Commit size comes from the commits API. The *first* dataset has 67,980 runs from 30 repositories (2025–26, 8.9% failures). The *confirmation* dataset was collected under its own pre-registered protocol: the first 30 eligible repositories from a star-sorted search, excluding every repository used before, with runs created from 15 August to 30 September 2026. That gives 78,576 runs (28.2% failures) after dropping 3,773 runs (4.6%) whose fork commits could no longer be fetched.

IV. A LEAKAGE-SAFE PROBLEM SETUP

Availability registry. Every column is recorded in a versioned registry with the stage at which it becomes known: pre-build, build start, or post-build. The predictor refuses to start if any model input is not marked available at prediction time, so leaky fields cannot re-enter by accident. Fields such as `tr_log_*` test counts, durations, and comment counts that may be written after the build are blocked.

Finish-time-safe history. A prior build contributes its outcome to a new build's history only if it *finished* strictly before the new build was triggered. Ties are ordered by finish time, trigger time, then build id. A brute-force recomputation rechecks the history values on sampled repositories in every run, and a deliberately leaked feature makes it fail. Optional *finish buffers* of 1 h and 6 h delay each outcome further.

TABLE II
THE SAME TASK UNDER PROGRESSIVELY STRICTER SETUPS (TRAVIS CI).

Setup	AUC
Random split, 31 features incl. post-build log fields	0.896
Random split, 13 strict pre-build features	0.690
Random split, 39 features with repository history (stacking)	0.912
Chronological split, job level, pooled (RF)	0.821
Walk-forward, build level, *within-repo* (Tier C RF)	0.628

How much the shortcuts inflate results. Table II shows the same family of models under progressively stricter setups. On a random split, adding post-build fields lifts AUC from 0.690 to 0.896. Even without leaky fields, the random-split AUC of a 39-feature model with repository history (0.91) falls to 0.82 on a chronological split and to 0.63 when measured *within* repositories at the build level. That last setup is the one that matches a team's decision, and it is the one we report.

V. METHOD

A. Feature tiers

Tier A holds 18 change and context features known at trigger time: source and test churn, files added, deleted and modified by type, time of day and week, pull request flag, core-team author, and default branch, plus the language. **Tier C** adds project context documented by TravisTorrent (team size, code size, test density, past commits on the touched files, commits in the build) and repository history: the previous finished build's outcome and the repository's last-5 and overall failure rates. On GitHub Actions the change features are the head commit's additions, deletions, files changed, message length and parents, with the workflow as a categorical feature.

Tier E adds two groups. *Stream history*: a stream is the build's pull request (on GitHub Actions, the PR author and head branch, since runs carry no PR number) or, for push builds, its branch. Stream features are the outcome of the stream's last finished build, its last-5 and overall failure rates, its count of finished builds and failure streak, the outcome of the build's predecessor (on Travis, the previous build on the branch; on GitHub Actions, the previous run of the same workflow on the stream), and an exponentially weighted repository failure rate. *In-flight context* records whether the previous build and the stream's last build are still running, how many repository builds are running, how many started in the last hour, and the hours since the stream last triggered. In-flight features use only the existence and start times of earlier builds, never their outcomes.

B. Model

Within-repository ranking is the goal, so Tier E combines a binary LightGBM classifier [16] with a LightGBM ranker trained with the XE-NDCG objective [17]. Each ranking query is one repository's builds in one calendar month. Let $b(x)$ be the classifier's log-odds and $r(x)$ the ranker's score. Both are standardised with *training-score* moments and averaged,

$$s(x) = \tfrac{1}{2}\,\frac{r(x)-\mu_r}{\sigma_r} + \tfrac{1}{2}\,\frac{b(x)-\mu_b}{\sigma_b}, \qquad (1)$$

and one global Platt map [18], fitted on training scores, turns $s(x)$ into a probability. Hyperparameters were fixed before any confirmation fold was scored. Explanations use SHAP values [19] of the binary half.

VI. Evaluation Protocol

Walk-forward folds. Builds are sorted by trigger time and split into 10 equal chronological folds. Fold k is tested after training on folds $1, \ldots, k-1$; test folds are 6–10. On Travis CI, every Tier E design choice was made on fold 6 alone and frozen in code, and folds 7–10 were scored once afterwards. On GitHub Actions the frozen recipe was applied unchanged.

Metrics. The primary metric is *macro within-repository AUC*: the AUC within each repository that has both outcomes in the test fold, averaged over repositories. We also report within-repository recall and lift when each repository reviews its riskiest 15% of builds ("recall@15%"), and pooled AUC for reference. Paired differences use a repository-cluster bootstrap with 1,000 resamples [13].

Controls. A global train-label shuffle drops the ranker to 0.48–0.50 within-repository AUC, confirming that the pipeline does not leak labels. We also report baselines that use only repository history.

Pre-registration. Each GitHub Actions test followed a protocol that was written, hashed with SHA-256, and committed before its data was collected [14]. Each fixed the decision rule in advance: the primary result is the mean over folds 6–10 of the paired ΔAUC (Tier E − Tier C). The rule *confirms* if the mean is positive and at least 4 of 5 fold CIs exclude zero, is *inconclusive* if the mean is positive otherwise, and *does not replicate* if it is not positive.

VII. Results

Fig. 1 summarises every evaluation.

A. Travis CI

Tier E raises macro within-repository AUC from 0.628 to **0.698** (Table III), with every confirmation fold's paired CI above zero (fold ΔAUC: +0.068, +0.067, +0.070, +0.074). Within-repository recall@15% improves by +0.068. Unlike a pure persistence effect, the gain *grows* as outcomes are withheld longer (Fig. 2): in-flight features remain informative when recent outcomes are not yet known.

Attribution (Table IV). On Travis the ranking objective is the largest single lever: it is worth +0.031 on Tier C features alone, and stream and in-flight features add about +0.03 more. Build-matrix size contributes only ∼0.003; the deployed variant omits it. The ranker trades pooled AUC for within-repository ordering: the blend's pooled AUC is 0.828 against 0.856 for Tier C RF. Teams that need pooled ranking should use the binary Tier E model, which beats Tier C on both metrics.

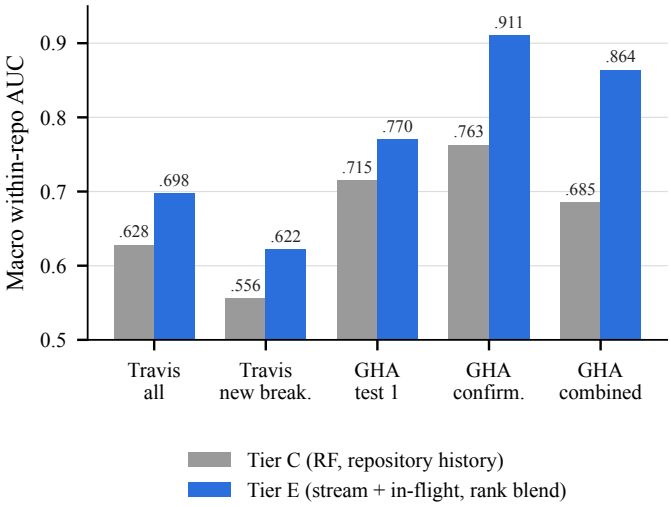


Fig. 1. Macro within-repository AUC of Tier C (Random Forest with repository history) and Tier E across all evaluations (test folds; 0 h finish buffer). Travis uses folds 7–10, GitHub Actions (GHA) folds 6–10.

TABLE III
Travis CI, folds 7–10: macro within-repo AUC by finish buffer. Every fold's paired 95% CI for ΔAUC is above 0.

Finish buffer	Tier C RF	Tier E blend	ΔAUC
0 h	0.628	**0.698**	+0.070
1 h	0.600	**0.681**	+0.081
6 h	0.585	**0.672**	+0.087

B. New breakages

Much of the all-builds signal comes from streams that are already failing: in folds 7–10, 55% of failures occur on streams whose last known outcome was a failure. To check that the model does more than persistence, we rescored the saved predictions on *new breakages*, builds whose previous build on the same stream passed. These are 25% of failures, and the builds fail only 6.6% of the time. Here the repository's last-5 failure rate is at chance (0.504), Tier C scores 0.556, and Tier E scores **0.622**. The gain (+0.066) is as large as on all builds, and every fold's CI is above zero at every buffer. The honest headline for fresh breakages is therefore ∼0.62, not 0.70. This analysis is post hoc.

C. GitHub Actions: two pre-registered tests

Test 1 (30 repositories): inconclusive. Tier E scored 0.770 against 0.715 (+0.055), positive in all five folds, but only 3 of 5 fold CIs excluded zero (the rule required 4), so the pre-registered verdict is *inconclusive*. Recall@15% was 0.48

TABLE IV
ATTRIBUTION ON TRAVIS CI (FOLDS 7–10, 0 H).

Variant	Within-repo	Pooled
Tier C, Random Forest	0.628	0.856
Tier C, binary LightGBM	0.633	–
Tier C, LightGBM ranker	0.664	–
Tier E, binary LightGBM	0.652	**0.881**
Tier E, LightGBM ranker	0.696	–
Tier E, blend	**0.698**	0.828
Tier E, blend without job counts	0.695	0.826

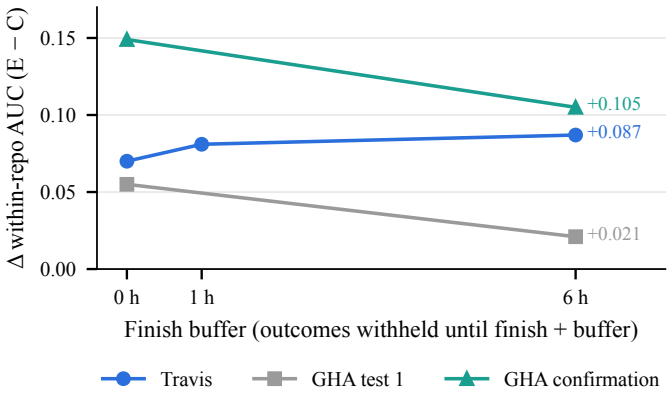


Fig. 2. Gain of Tier E over Tier C as outcomes are withheld longer. On Travis the gain grows; on GitHub Actions, where most runs finish within an hour, a 6 h delay removes much of the recent stream signal.

against 0.37 (4 of 5 folds), and the gain shrank to +0.021 with a 6 h buffer.

Confirmation (30 new repositories): confirms. With every repository, the period and the protocol fresh, Tier E scored **0.911** against 0.763 (+0.149). All five fold CIs exclude zero, and the smallest lower bound is +0.090 (Fig. 3, Table V). Recall@15% rose from 0.31 to 0.43 (lift 2.82× vs. 2.04×), and Tier E beat Tier C in all 24 test repositories. The secondary analysis on both datasets combined (60 repositories) gave +0.179, again 5 of 5 folds.

Is it only "it failed last time"? The confirmation gain is nearly three times test 1's, and failures are three times as common. We therefore checked a naive persistence score: whether the stream's previous finished run failed. It reaches only ∼0.72, below Tier C (Fig. 4), and the per-repository gain is uncorrelated with the repository's failure rate ($r = 0.03$). This check was exploratory, not pre-registered.

What differs from Travis. On GitHub Actions the stream and in-flight features carry the gain; the ranking objective alone adds only +0.010 (test 1) and +0.042 (confirmation). Most runs finish within an hour, so the very recent outcome of the same stream is the key signal, and withholding it for 6 h removes most of the new-breakage gain. The size of the gain varies by project (+0.055 to +0.149); the consistent finding is its sign.

TABLE V
GITHUB ACTIONS CONFIRMATION (30 NEW REPOSITORIES, FOLDS 6–10).

Setting	Tier C	Tier E	ΔAUC	CIs > 0
All runs, 0 h (primary)	0.763	**0.911**	+0.149	5/5
Label-validity sensitivity	0.765	0.911	+0.147	5/5
New breakages, 0 h	0.642	0.683	+0.041	2/5
All runs, 6 h buffer	0.741	0.846	+0.105	5/5
New breakages, 6 h	0.637	0.634	−0.004	0/5
Combined, 60 repos (secondary)	0.685	0.864	+0.179	5/5

D. Negative results

Adding repository-level streak and gap features to Tier C *reduced* within-repository AUC by 0.019 (4 of 5 folds significantly). Streaks that mix all of a repository's branches are noise; per-stream history is the version of this idea that helps.

VIII. DEPLOYED SYSTEM

Architecture (Fig. 5). A FastAPI service receives signed GitHub `workflow_run` events. When a run is *requested*, the service builds its stream history and in-flight context from the repository's stored runs and fetches its commit's size. It then scores the run with the Tier E model, trained on folds 1–9 of the 146,556 GitHub Actions runs, and stores a shadow prediction. When the run *completes*, its outcome is attached. Other conclusions, such as `cancelled`, are stored but never become history, mirroring training. Runs are timed by GitHub's own `created_at` and `updated_at`, so a late webhook delivery does not change a run's history. Tests check that the live history equals the offline training pass row by row. The deployed artifact reproduces the research fold-10 predictions exactly, with a maximum difference of 0. On that holdout its within-repository AUC is 0.913, and 0.893 with all workflow identities treated as unseen, as for a newly connected repository.

Per-repository alert budget. A single global threshold does not implement the evaluated metric. Taken as the 85th percentile of the holdout, which was 36% failures, it sat at 0.94 and flagged between 3% and 51% of a repository's runs. Alerts instead follow the evaluation: a run is flagged when its risk reaches the 85th percentile of its repository's last 100 scored runs. Until a repository has 20 scored runs, it uses a "typical repository" threshold: the median, over holdout repositories, of each one's 85th percentile. Neither threshold uses labels. Replayed in time order on the holdout (Fig. 6, Table VI), each repository flags 11–22% of its runs, and the mean share of a repository's failures caught rises from 34% to 41%, close to the 43% measured in hindsight. Pooled precision falls (0.80 vs. 0.97) only because the global threshold spends nearly all its alerts on the repositories that fail most.

Dashboard and public demonstration. A React dashboard shows risk trends, the risk distribution, model evidence (Fig. 7), and per-repository risk timelines with real outcomes (Fig. 8). It also lets users score a run with the history the server derives (Fig. 9) and explore what-if changes with SHAP explanations (Fig. 10). A read-only public deployment replays

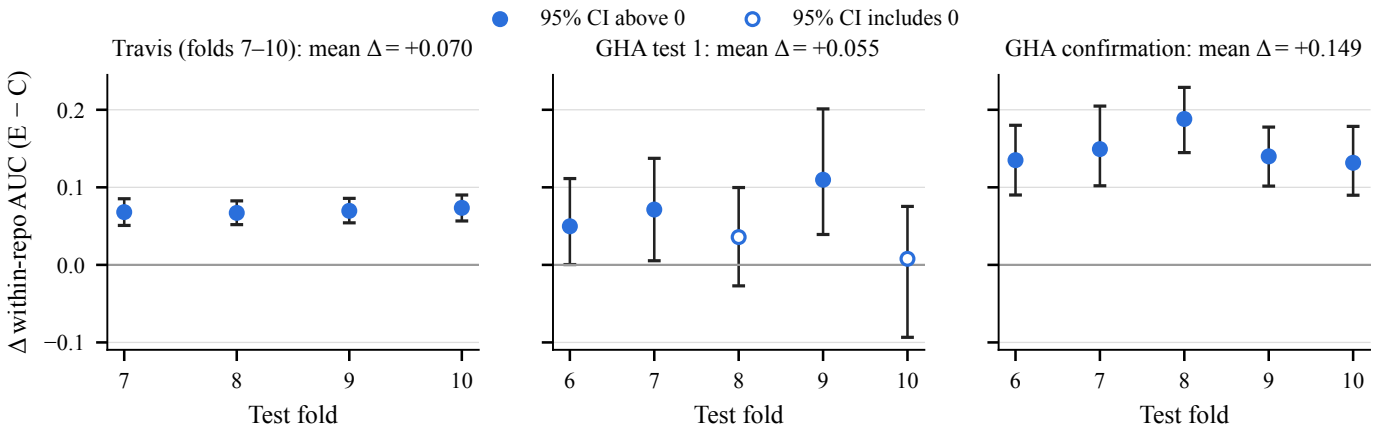


Fig. 3. Per-fold paired ΔAUC (Tier E − Tier C) with repository-bootstrap 95% CIs. Filled markers: CI excludes 0. The confirmation test meets the pre-registered rule (at least 4 of 5) in all five folds; test 1 met it in three.

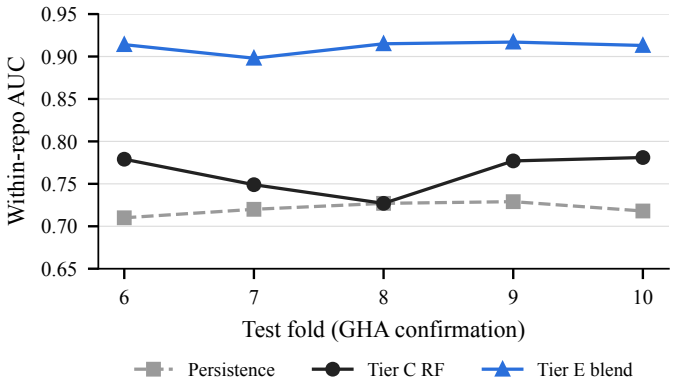


Fig. 4. Confirmation test: Tier E against Tier C and a naive stream-persistence score (exploratory).

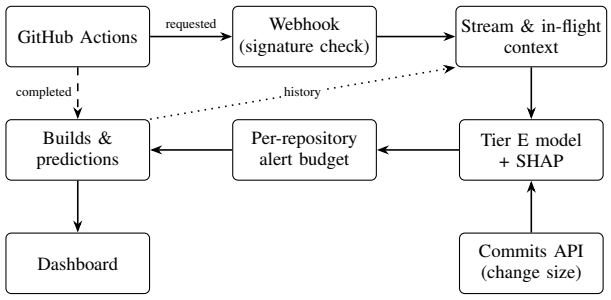


Fig. 5. Scoring path. A *requested* workflow run is scored before it executes; its *completed* event attaches the real outcome, which becomes history for later runs. Nothing blocks CI.

450 real runs from `twbs/bootstrap`, `freeCodeCamp` and `yt-dlp` through the real webhook, at their real GitHub times. The runs come from the model's holdout window and were never trained on. PR authors and branch names are pseudonymised. Its within-repository AUCs (0.80, 0.95, 0.74) are close to the research evaluation of the same runs (0.78, 0.97, 0.79). The remaining gap comes from the demonstration starting each repository with empty history.

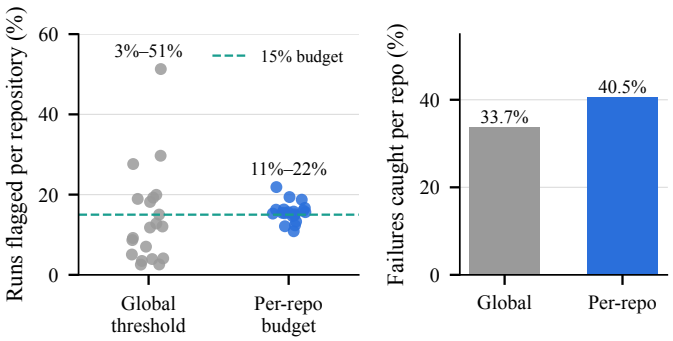


Fig. 6. Deployed model's holdout: share of each repository's runs flagged (left; one dot per repository) and mean share of a repository's failures caught (right), for one global threshold and for the per-repository budget.

TABLE VI
ALERT POLICIES ON THE DEPLOYED MODEL'S HOLDOUT (20 REPOSITORIES).

	Global (0.94)	Per-repo budget
Runs flagged per repository	3–51%	11–22%
Failures caught per repository (mean)	34%	**41%**
Pooled precision of alerts	0.97	0.80

IX. Threats to Validity

Internal. The new-breakage analysis and the persistence check are post hoc. Travis fold 6 was used for development, so only folds 7–10 are confirmatory. Finish times on Travis are derived from job durations with a fallback. On GitHub Actions, 4.6% of confirmation runs were lost to unfetchable fork commits, and these runs failed more often (38.5% vs. 28.2%).

External. Repositories come from popular open-source projects. In the confirmation data, the cap of the earliest 4,000 runs per repository left six of the busiest projects in training folds only. The gain's size varies widely by project; we claim its sign, not a fixed size.

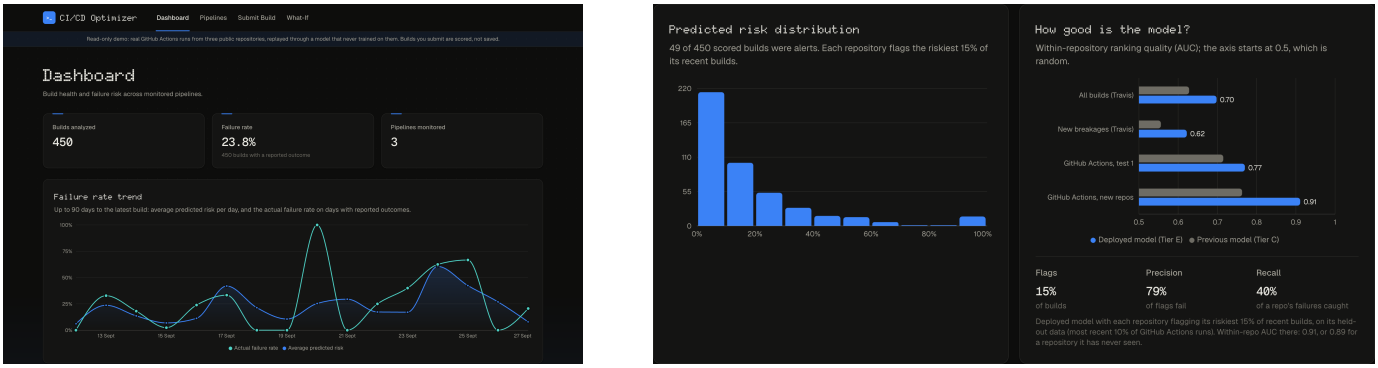


Fig. 7. Dashboard of the public demonstration. Left: summary and daily failure-rate trend; average predicted risk tracks the actual failure rate. Right: risk distribution of the 450 scored runs (49 alerts under the per-repository budget) and the model-evidence panel with the research results and the deployed model's holdout statistics.

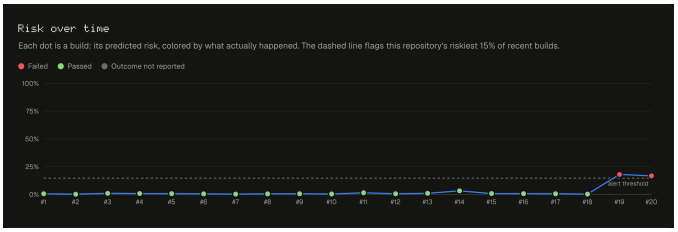


Fig. 8. Risk timeline for `twbs/bootstrap`: each dot is a run, coloured by its real outcome. The dashed line is this repository's alert threshold under its 15% budget. The two newest runs, both real failures, rise above it.

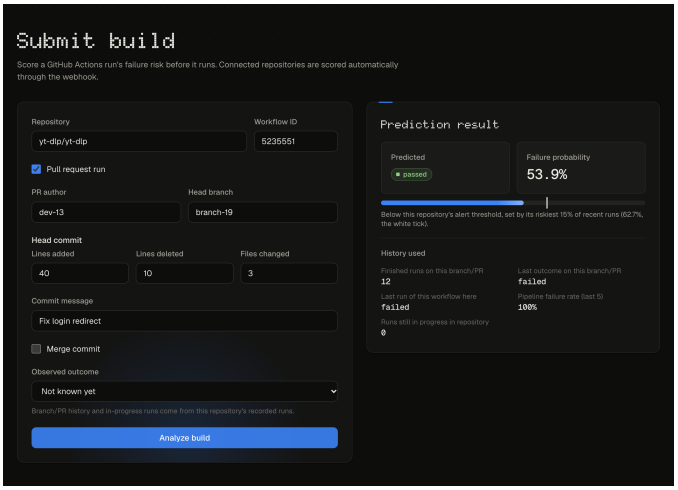


Fig. 9. Scoring a pull-request run on a failing `yt-dlp` stream. The server derives the history (12 finished runs, last one failed) and compares the risk with the repository's own alert threshold.

Construct. Within-repository AUC measures ranking, not calibrated risk. Recall at a 15% budget assumes teams review a fixed share of builds. A run on GitHub Actions is one workflow, so one push yields several runs that share history.

X. Pre-Registered Forward-Looking Test

All results above test the *recipe* on data that already existed. To test the *deployed system*, we pre-registered a forward-

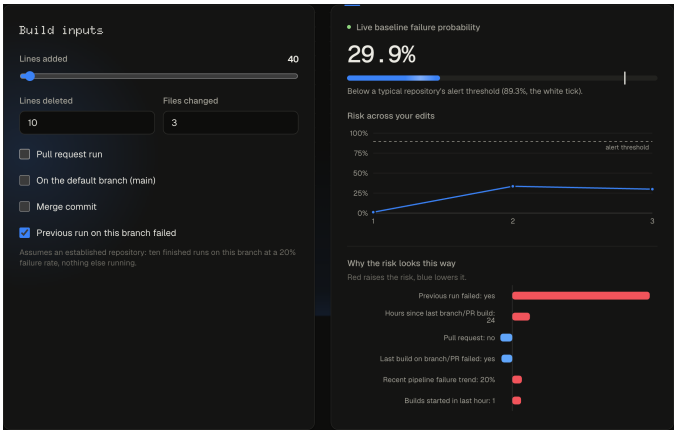


Fig. 10. What-if page: SHAP contributions of the binary half explain the live risk. Here, whether the previous run of the same workflow failed dominates.

looking evaluation before collecting any of its data. It scores October 2026 runs from the 60 training repositories, collected no earlier than 28 November 2026, with the frozen model (artifact SHA-256 `57c26f17...`) and the frozen alert rule. History starts empty on 1 October, exactly as if the service had been connected then. Each repository's first 25% of runs serves as warm-up and is not evaluated. The primary result is the macro within-repository AUC against a stream-persistence baseline, with a repository-bootstrap CI. The system "holds up" if the lower CI bound is above zero *and* the AUC is at least 0.80. The protocol, repository list, and fingerprints of the frozen code are committed with the project.

XI. Conclusion

Build failure prediction looks far easier than it is when leaky fields, random splits, or pooled metrics are allowed: on Travis CI, an AUC of 0.91 shrinks to 0.63 under a protocol that matches a team's decision. Within that protocol, stream-level history, in-flight context, and a ranking objective improve within-repository ranking on Travis CI (0.628 to 0.698, and 0.556 to 0.622 on new breakages). The gain carried over to GitHub Actions, where a pre-registered confirmation on 30

new repositories met its decision rule in every fold. Deployed with a per-repository review budget, the model catches about 41% of a repository's failures in its riskiest 15% of runs. The forward-looking test will show whether the frozen system holds up on data that did not exist when it was built.

AVAILABILITY

Code, protocols, and results: https://github.com/yasaswinipakam/cicd-ai-optimizer. Read-only demonstration: https://cicd-ai-optimizer.onrender.com.

REFERENCES

[1] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in *Proc. 31st IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2016, pp. 426–437.

[2] F. Hassan and X. Wang, "Change-aware build prediction model for stall avoidance in continuous integration," in *Proc. ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)*, 2017, pp. 157–162.

[3] A. Ni and M. Li, "Cost-effective build outcome prediction using cascaded classifiers," in *Proc. 14th Int. Conf. Mining Software Repositories (MSR)*, 2017, pp. 455–458.

[4] J. Xia and Y. Li, "Could we predict the result of a continuous integration build? An empirical study," in *Proc. IEEE Int. Conf. Software Quality, Reliability and Security Companion (QRS-C)*, 2017, pp. 311–315.

[5] B. Chen, L. Chen, C. Zhang, and X. Peng, "BuildFast: History-aware build outcome prediction for fast feedback and reduced cost in continuous integration," in *Proc. 35th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2020, pp. 42–53.

[6] X. Jin and F. Servant, "A cost-efficient approach to building in continuous integration," in *Proc. 42nd ACM/IEEE Int. Conf. Software Engineering (ICSE)*, 2020, pp. 13–25.

[7] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[8] M. Beller, G. Gousios, and A. Zaidman, "TravisTorrent: Synthesizing Travis CI and GitHub for full-stack research on continuous integration," in *Proc. 14th Int. Conf. Mining Software Repositories (MSR)*, 2017, pp. 447–450.

[9] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.

[10] T. Rausch, W. Hummer, P. Leitner, and S. Schulte, "An empirical analysis of build failures in the continuous integration workflows of Java-based open-source software," in *Proc. 14th Int. Conf. Mining Software Repositories (MSR)*, 2017, pp. 345–355.

[11] K. Gallaba, C. Macho, M. Pinzger, and S. McIntosh, "Noise and heterogeneity in historical build data: An empirical study of Travis CI," in *Proc. 33rd ACM/IEEE Int. Conf. Automated Software Engineering (ASE)*, 2018, pp. 87–97.

[12] T. Durieux, R. Abreu, M. Monperrus, T. F. Bissyandé, and L. Cruz, "An analysis of 35+ million jobs of Travis CI," in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, 2019, pp. 291–295.

[13] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[14] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. National Academy of Sciences*, vol. 115, no. 11, pp. 2600–2606, 2018.

[15] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[16] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[17] S. Bruch, "An alternative cross entropy loss for learning-to-rank," in *Proc. Web Conference (WWW)*, 2021, pp. 118–126.

[18] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*. MIT Press, 1999, pp. 61–74.

[19] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.